1772 (errors:1) words

```
File: main.tex
Encoding: utf8
Sum count: 1772
Words in text: 1398
Words in headers: 40
Words outside text (captions, etc.): 276
Number of headers: 16
Number of floats/tables/figures: 10
Number of math inlines: 31
Number of math displayed: 1
Subcounts:
  text+headers+captions (#headers/#floats/#inlines/#displayed)
  0+15+0 (1/0/0/0) _top_
  198+1+0 (1/0/0/0) Section: Introduction
  44+2+0 (1/0/0/0) Section: Technical Approach
  45+2+0 (1/0/1/0) Subsection: Particle Filter
  87+2+0 (1/0/0/0) Subsection: Motion Model
  215+2+41 (1/2/14/0) Subsection: Sensor Model
  122+1+0 (1/0/1/0) Subsection: Resampling
  71+2+0 (1/0/4/1) Subsection: Pose Estimate
  63+2+0 (1/0/0/0) Section: Experimental Evaluation
  152+2+132 (1/3/11/0) Subsection: Simulation Testing
  122+3+103 (1/5/0/0) Subsection: On Car Testing
  190+1+0 (1/0/0/0) Section: Conclusion
  0+2+0 (1/0/0/0) Section: Lessons Learned
  45+1+0 (1/0/0/0) Subsection: Ana
  44+1+0 (1/0/0/0) Subsection: Brianna
  0+1+0 (1/0/0/0) Subsection: Rohan


(errors:1)
```

Lab # 5 Report: Localization and Pose Determination Using Randomized Particle Filters with Monte Carlo Methods

Team # 3

Rohan Das
Brianna Lebrun
Ana Tejeda

Robotics: Science & Systems

April 11, 2026

1 Introduction

Autonomous systems rely on accurate localization to safely navigate their environment. In this lab, we address the problem of estimating a racecar's pose (position and orientation) when its initial location is unknown within a known map. The challenge is sensor measurements are noisy, motion is uncertain, and errors in estimation can accumulate over time.

Our goal is to develop a localization system that not only produces accurate pose estimates, but is also robust to noise and capable of recovering from significant errors. In practice, this means the system must continuously track the vehicle's position as it moves and maintain stability even when sensor readings or motion estimates are unreliable.

To solve this problem, we implement Monte Carlo Localization (MCL), a particle filter method that represents uncertainty using a distribution of possible poses rather than a single estimate. This approach is particularly well suited for scenarios with unknown initial conditions and non-Gaussian noise because it can model multiple hypotheses and converge to the correct pose over time.

We evaluate our implementation both qualitatively, through visualization of particle behavior, and quantitatively using simulation and real-world experiments, with the goal of minimizing error between the estimated and true pose.

2 Technical Approach

We implement Monte Carlo Localization, which is a particle-filter based approach relying on a motion model and a sensor model. One of its benefits is that it is one of the most versatile filtering algorithms, making no assumptions about initial location or Gaussian noise.

2.1 Particle Filter

Rather than making a deterministic prediction of the pose, Monte Carlo Localization initializes a vast collection of particles (say of size N) throughout the map. Then, the particles are weighted based on a dynamic perceived likelihood, and the pose estimate is a weighted average of them.

2.2 Motion Model

The motion model moves the particles along with the car. Specifically, as the car moves, we use the velocity commands to compute real-world deltas for each particle and move them accordingly. Then, Gaussian noise is added to the particles, proportional to their odometry magnitude so high-speed particles still have significant noise. We also implemented a "deterministic" version that doesn't add this noise for the sake of unit tests and basic demonstrations, but the addition of noise is crucial for the model to be effective in real environments.

2.3 Sensor Model

The sensor model is the evaluation component of the particle filter, which uses the LiDAR data to judge how likely a particle pose is. This evaluation is made based on each beam according to four cases:

- p_{hit}, weighted at 74%: in this case, the beam reflected normally. If the beam's length is d, we expect a true value of d, so our distribution is Gaussian with mean d.
- p_{short}, weighted at 7%: the sensor encounters an obstacle somewhere before the wall and is too short. This is a decreasing distribution from 0 to d.
- p_{max}, weighted at 7%: the beam can't exceed the sensor range of z, so this distribution is an indicator of the reading being $z_{max} = 200$.
- p_{rand}, weighted at 14%: a uniform distribution. This disregards any LiDAR reading to account for errors or issues we can't foresee.

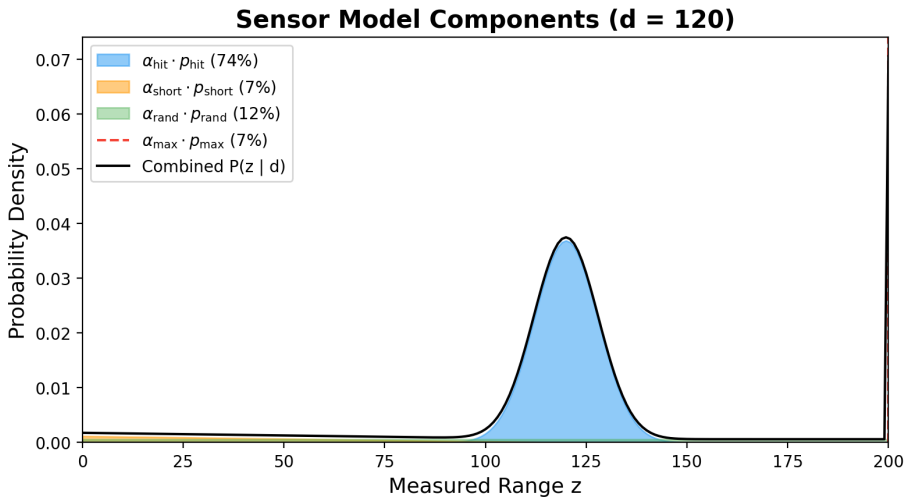

Figure 1: The aggregate probability density function is the weighted average of the distributions from each of the four cases.

Taking a weighted average of the distributions across each case, the sensor model converts an individual LiDAR beam into a probability distribution. Multiplying these distributions across all beams, we obtain the overall belief and normalize it to the final probability distribution. To avoid float underflow in python, we take the log sum: instead of multiplying hundreds of tiny p_i's, we compute $\exp(\sum_i \log(p_i))$. Another optimization we make is to use a precomputed lookup table for all possible ranges, to save the computation time for the hundreds of particles.

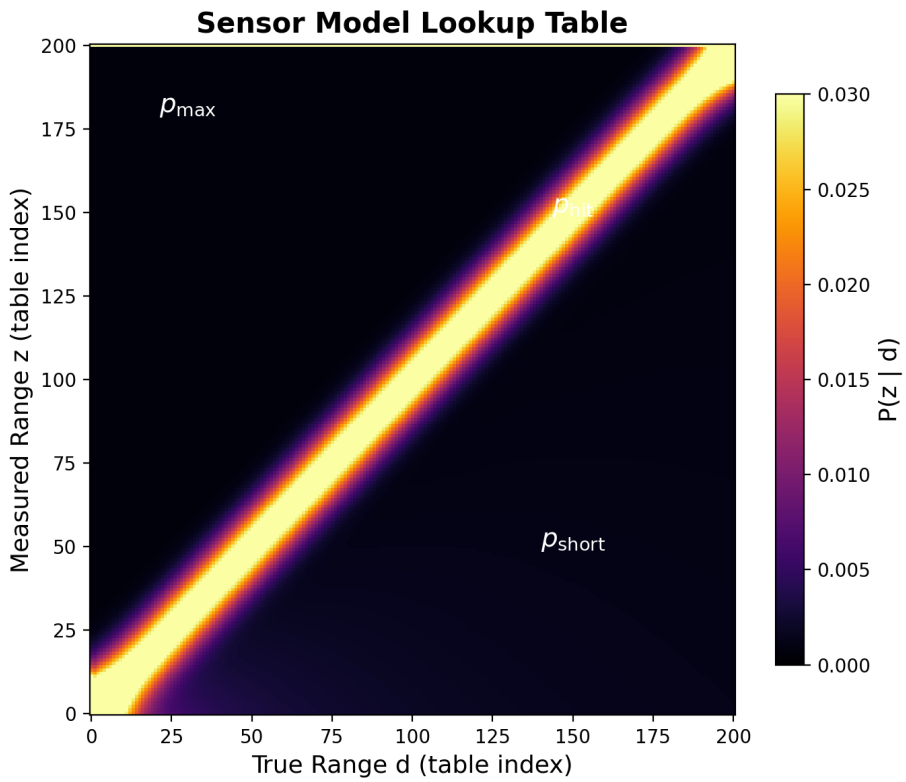

Figure 2: A heatmap of the sensor model's precomputed lookup table. The dominant trend is around $z = d$, which is the baseline for the 74%-weighted p_{hit} case.

2.4 Resampling

So far, our particle filter can move the particles and compute the likelihood of each. The next step of Monte Carlo Localization is to use this distribution to resample N particles from the sensor model's discrete belief space. Overall, this duplicates particles with high belief and removes the unlikelier ones.

A common risk in particle filtering algorithms is "particle death", when multiple particles collapse onto the same trajectory. This defeats the purpose of the particle filter, which is to track hundreds of particles to capture a broad distribution. Therefore, upon resampling, we add a small Gaussian noise to each particle to maintain diversity. This noise causes a slightly larger error in simulation tests, but practically makes the localization more robust in the long-term.

2.5 Pose Estimate

Monte Carlo Localization uses a weighted average of the particle filter to compute the final pose estimate, where particles are weighted according to the sensor model. For a list of particles with locations (x_i, y_i, θ_i) with weights w_i, the position estimate is simply $\sum_i w_i(x_i, y_i)$. For the angle, we use the circular mean

$$\theta = \text{arctan2}(\sum_i w_i \sin_i, \sum_i w_i \cos_i).$$

This avoids any wraparound issues at an angle of π, and more faithfully represents the estimate as a weighted average of the orientation vectors.

3 Experimental Evaluation

For this lab, our goal was to minimize the error between the racecar's actual location and its predicted location. To achieve this, we ran several tests, both on the car and in simulation, to determine the parameters that would produce the lowest error. The brunt of this tuning occurred during the simulation phase, with minor adjustments being necessary after on-car tests were performed.

3.1 Simulation Testing

We evaluated the robustness of our particle filter in simulation using a controlled circular trajectory at constant speed. This setup provides a known ground-truth path while isolating localization performance under varying levels of odometry noise.
To simulate real-world uncertainty, Gaussian noise was injected into the odometry inputs under three conditions:

- **Baseline:** no injected noise
- **Low noise:** $\sigma_v = 0.1\,\text{m/s}$, $\sigma_\omega = 0.05\,\text{rad/s}$
- **High noise:** $\sigma_v = 0.3\,\text{m/s}$, $\sigma_\omega = 0.15\,\text{rad/s}$

Performance was measured using position and heading error. Position error is computed as the Euclidean distance between estimated and ground-truth position, while heading error is defined as the smallest angular difference between estimated and true orientation.

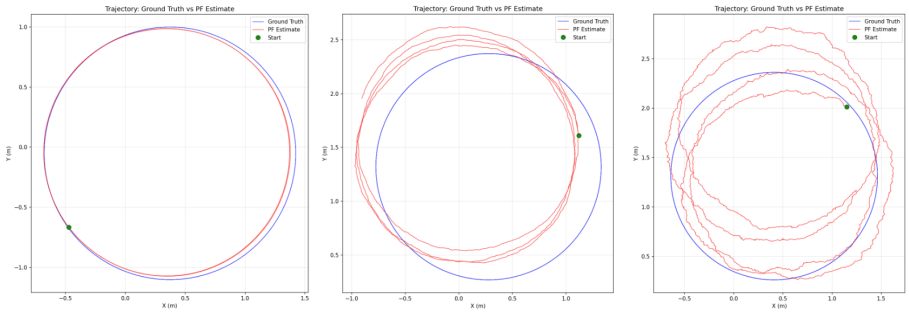

Figure 3: **Ground truth vs. particle filter trajectory under increasing noise.** The true trajectory is shown in blue and the particle filter estimate in red.The estimate closely tracks the true trajectory with no noise, remains stable with a slight offset under low noise, and diverges under high noise, indicating loss of localization.

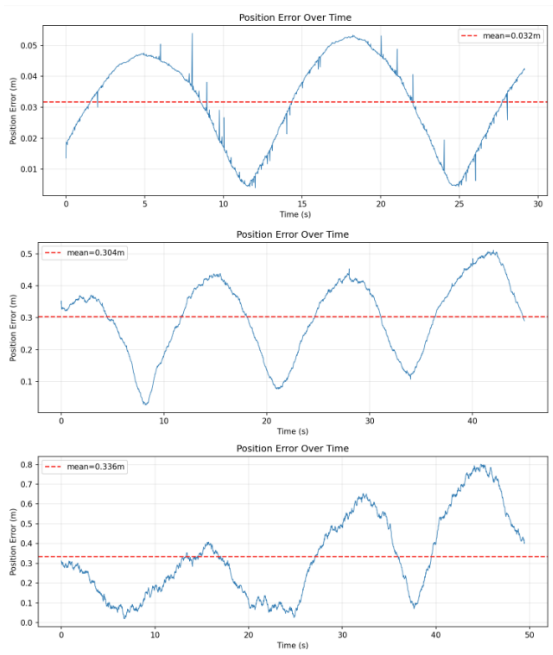

Figure 4: **Position error over time under increasing odometry noise.** With no injected noise, error stays under 5 cm; under low noise, it remains bounded below 50 cm; and under high noise, the mean reaches 34 cm and increases over time, indicating divergence.

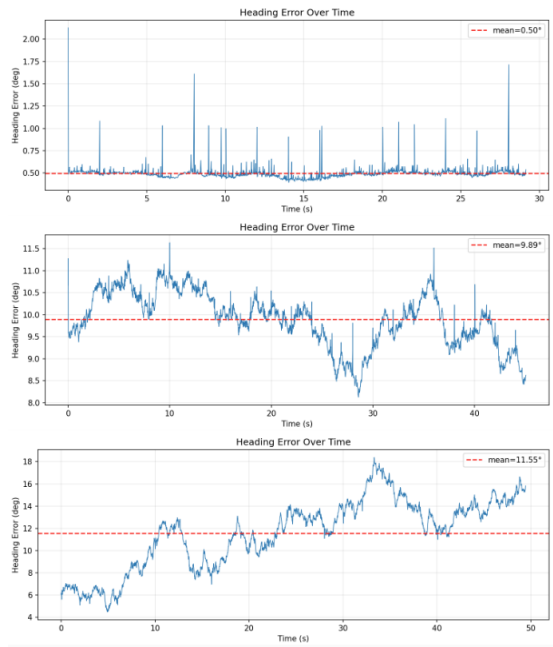

Figure 5: **Heading error over time under increasing odometry noise.** With no noise, error stays under 2° (mean 0.5°); under low noise, it remains bounded below 11.5° (mean 9.9°); and under high noise, the mean reaches 11.6° and increases over time, indicating divergence.

Overall, these results show that the particle filter works well under normal levels of noise, with both position and heading error remaining bounded over time. However, as odometry noise increases, the filter becomes increasingly unable to correct for accumulated motion error, leading to divergence. Based on this tradeoff, we selected $\sigma_v = 0.1\,\mathrm{m/s}$ and $\sigma_\omega = 0.05\,\mathrm{rad/s}$ as a balance between robustness and accuracy.

3.2 On Car Testing

To determine how much additional tuning was necessary after simulation, a series of tests were run to evaluate the effectiveness of our algorithm across different parameters. To make these comparisons, we used the ground truth estimate of the racecar's location.

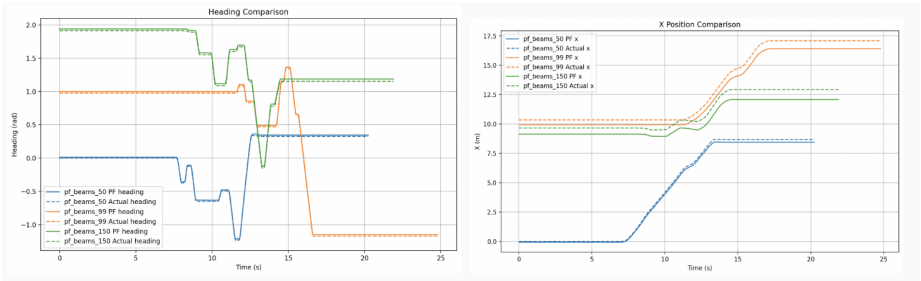

Figure 6: The error in heading and X position for beam count equal to 50, 99, or 150.

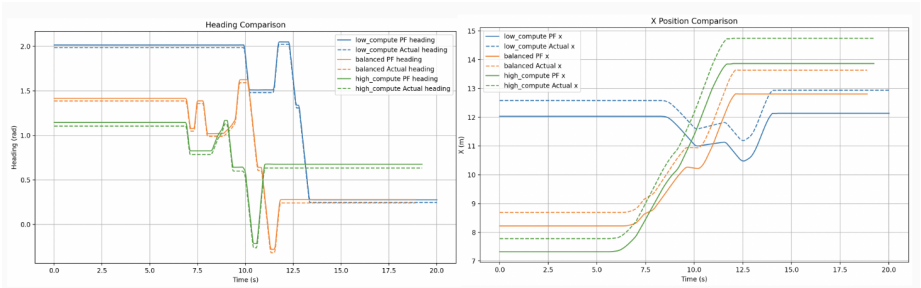

Figure 7: The error in heading and X position for combined variations in particle count and LiDAR beam count, comparing low_compute (50 particles, 50 beams), balanced (100 particles, 99 beams), and high_compute (200 particles, 150 beams) configurations.

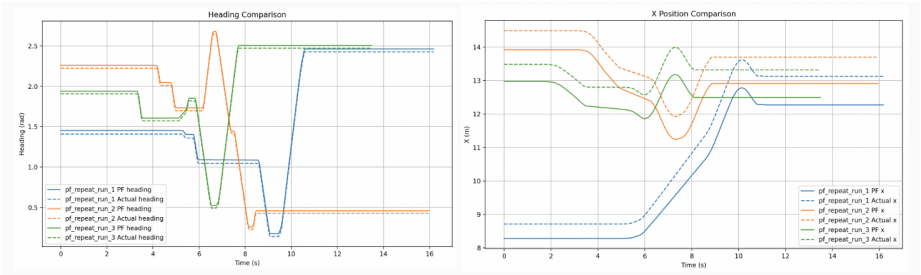

Figure 8: The error in heading and X position fo repeated parameters (particle count = 100, beam count = 99).

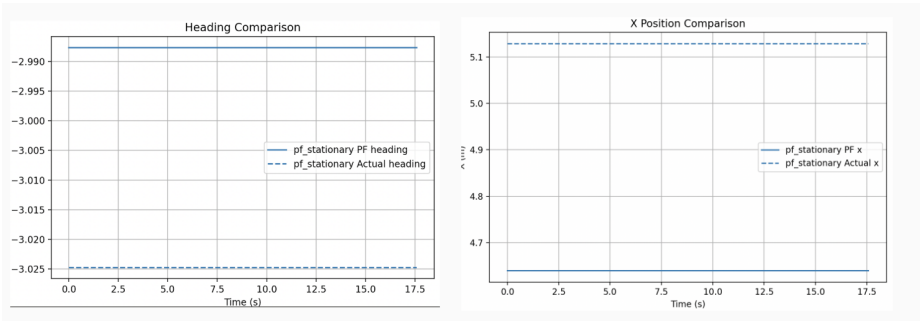

Figure 9: The error in heading and X position when the racecar is stationary (particle count = 100, beam count = 99).

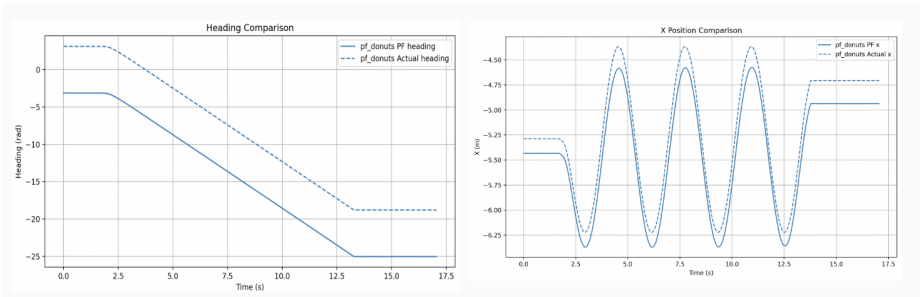

Figure 10: The error in heading and X position under circular motion (particle count = 100, beam count = 99).

From these test, we see that the error is minimized when the number of components involved in the localization algorithm is minimized. This is due to the added noise produced by these extra components.
Additionally the test show us the the error for the headings tends to fall within the range 0.02-0.04 and for the x-position, 0.5-0.7. While we could attempt to inject less noise or tune the probability parameters, with our initial setting we have already achieved a minimized localization error.

4 Conclusion

In this lab, we implemented and evaluated a Monte Carlo Localization (MCL) system for estimating the pose of a racecar using LiDAR and odometry data. Through both simulation and real-world experiments, we examined the effects of noise, particle count, and LiDAR beam density on localization performance. Our results show that the particle filter provides accurate and reliable pose estimates across a range of conditions. In simulation, the filter maintained low error under moderate odometry noise and only degraded under extreme noise

levels. In real-world experiments, localization remained consistent across all tested configurations, with position error typically between 0.5–0.7 meters and heading error between 0.02–0.04 radians.
Notably, increasing the number of particles and LiDAR beams did not significantly improve accuracy, indicating that the baseline configuration is already sufficient for robust performance. This suggests that increasing computational complexity provides minimal additional benefit in this setting.
While a small positional offset was observed during stationary tests, the filter remained stable over time and did not exhibit drift or divergence. Overall, these results demonstrate that the implemented MCL system is both effective and robust, providing dependable localization without requiring excessive computational complexity.

5 Lessons Learned

5.1 Ana

I learned that it's important not to just guess parameter values, but to systematically test and tune them to understand what actually works best. I also found that getting the system working well in simulation made the transition to the real car much smoother.

5.2 Brianna

I learned the importance of planning out all your tests and the reason why you need them long before you actually perform them. Also, I learned that's its really important to clearly define exactly what everyone is working on so there are clear expectations.

5.3 Rohan